Governing Coding-Agent Executions: An Agent-Independent Reference Model and Empirical Evaluation

Ricken Bazolo
Independent Researcher

Pre-results manuscript - experimental results intentionally pending

Abstract

Coding agents have evolved from code-completion systems into software-engineering actors capable of inspecting repositories, modifying multiple files, executing tools, running tests, and producing complete software changes. This increased implementation capability shifts part of the software-engineering problem from code generation toward the engineering system surrounding agent execution: context selection, policy enforcement, execution state, risk-bounded autonomy, verification, evidence, and human accountability.

Existing research and operational systems already address many of these concerns independently. Coding-agent harnesses provide tools, context, state, permissions, and execution environments; specification-driven approaches structure developer intent; process frameworks introduce staged workflows and verification; and recent work proposes deterministic control planes, risk-adaptive processes, and evidence-oriented governance. However, these mechanisms are typically distributed across different control surfaces.

This paper proposes the **AI Engineering Factory (AIEF) Reference Model**, an agent-independent model in which a **Governed Engineering Execution** is the central unit of control. The model separates probabilistic coding-agent reasoning from deterministic engineering control and composes organizational constraints, project knowledge, task-specific risk, bounded autonomy, execution state, workspace binding, verification, evidence, and accountability into an execution-specific control contract.

AIEF is provided as an executable reference implementation. A controlled empirical study is designed to compare four levels of engineering control, ranging from direct coding-agent execution to fully governed execution, and to measure policy compliance, reproducibility, traceability, evidence completeness, verification reliability, and the associated computational and human cost.

[Experimental-results statement to be inserted after completion of the study.]

Keywords: coding agents, agentic software engineering, software engineering governance, AI harness engineering, engineering control plane, software agents, verification, traceability, human-in-the-loop.

1. Introduction

Large language models have rapidly evolved from code-generation assistants into coding agents capable of interacting directly with software repositories and development environments. SWE-agent demonstrated that an agent-computer interface can materially affect an agent's ability to navigate repositories, modify files, and execute software-engineering tasks [1]. OpenHands subsequently generalized this interaction model through sandboxed execution, command-line access, browsing, multiple agent implementations, and benchmark integration [2].

This transition is visible in software-development practice. A large-scale study of 128,018 GitHub projects estimated coding-agent adoption at approximately **22.20% - 28.66% by February 21, 2026**, illustrating how rapidly agents moved from research prototypes into real development workflows [3].

Coding agents differ fundamentally from conventional code-completion tools. An agent may receive an engineering objective, inspect an unfamiliar repository, search for relevant files, modify code, execute commands, interpret failures, revise its solution, and produce a final software change.

The relevant unit of engineering therefore changes from an isolated generated fragment to an **execution trajectory**.

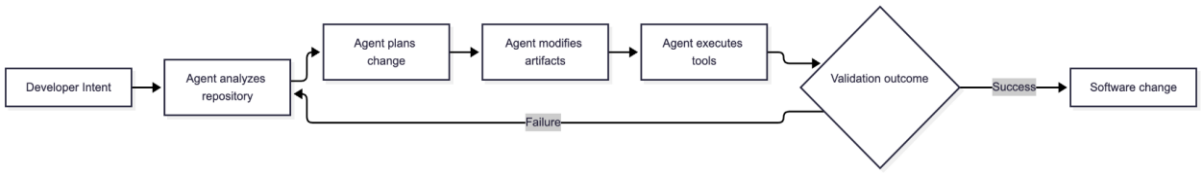


Figure 1. From isolated code generation to an engineering execution trajectory.

Repository-level software engineering further increases the complexity of this trajectory. SWE-bench established real-world issue resolution as an evaluation problem in which tasks may require coordinated changes across multiple functions, classes, files, and environmental dependencies [4].

As coding agents gain implementation capability, an increasing share of engineering reliability depends on the surrounding system. The important questions become not only whether an agent can produce code, but also:

- which context it receives;
- which constraints apply;

- which actions it is authorized to perform;
- how risk affects autonomy;
- how execution state survives sessions and agents;
- how claims are independently verified;
- which evidence remains after the execution;
- where human responsibility is retained.

Recent research increasingly recognizes this shift. Agentic Software Engineering proposes moving beyond isolated code generation toward goal-oriented agents operating across structured engineering processes and interacting bidirectionally with human engineers [5].

At the runtime level, AI Harness Engineering argues that software-engineering capability emerges from the interaction among model, harness, and environment rather than from model capability alone. Its responsibilities include task specification, context selection, project memory, task state, observability, verification, permissions, and intervention recording [6]. Harness Engineering independently characterizes production coding agents as platforms comprising execution loops, tools, context management, safety controls, orchestration, and extension surfaces [7].

Governance is evolving in parallel. Rel(AI)Build introduces a deterministic control plane above coding harnesses for configuration governance, permissions, process gates, and traceability [8]. Agentic Agile-V combines structured engineering artifacts, risk-adaptive workflows, evidence bundles, independent verification, and human approval [9]. MAGE emphasizes durable engineering representations, bounded action, validators, gates, independent evaluation, and retained human authority [10].

Consequently, most individual mechanisms required for governed agentic development already exist.

The unresolved problem is increasingly one of **composition**.

Context may be stored in repository instructions. Permissions may belong to the coding-agent runtime. Specifications may belong to a project methodology. Approval requirements may live in organizational configuration. Validation may occur in CI. Execution history may remain inside one agent session.

Each mechanism may operate correctly while the complete engineering execution remains fragmented.

This paper investigates the following proposition:

> **Agentic software engineering can be modeled as the interaction between a probabilistic execution plane and a deterministic engineering control plane responsible for context, policy, risk, autonomy, state, verification, evidence, and accountability.**

We introduce the **AIEF Reference Model**, in which the central object is not the coding agent itself but the **Governed Engineering Execution**.

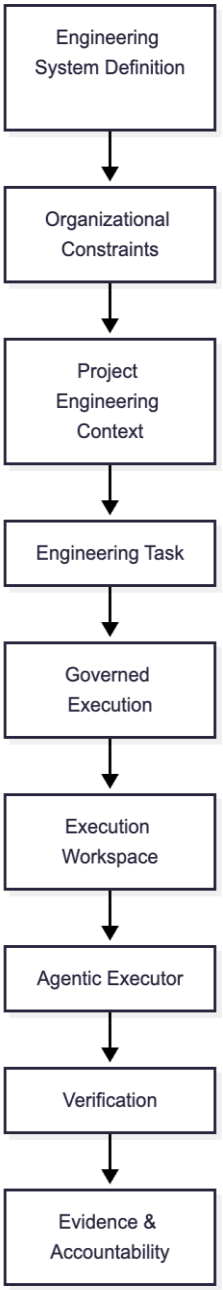


Figure 2. High-level AIEF Reference Model.

The model deliberately separates engineering semantics from any specific coding agent, model, source-control provider, programming language, or workspace technology.

This work makes four contributions.

1. A structured taxonomy of engineering-control responsibilities across coding-agent harnesses, specification-oriented systems, process frameworks, and deterministic control-plane approaches.
2. An implementation-independent Reference Model separating deterministic engineering control from probabilistic agent reasoning.
3. AIEF as an executable artifact implementing this model across heterogeneous coding agents.
4. A controlled empirical design evaluating both the benefits and the cost of progressively increasing engineering control.

The study therefore does not ask whether coding agents can generate software changes.

It asks:

> **What happens when coding-agent execution itself becomes an explicitly governed engineering process?**

2. Background

2.1 Coding agents

Traditional generative coding systems primarily returned text or source-code suggestions to a developer who remained directly responsible for applying and validating them.

Coding agents introduce an iterative interaction loop.

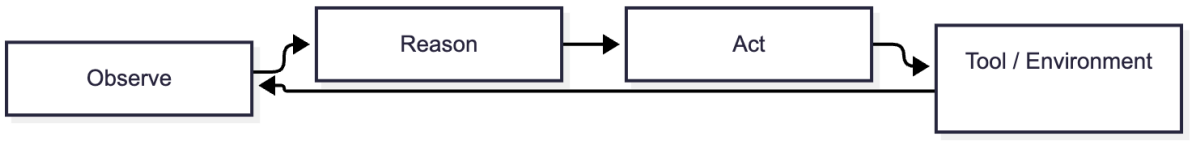


Figure 3. Simplified coding-agent interaction loop.

SWE-agent demonstrated that the interface between agent and computer environment materially influences software-engineering performance [1]. OpenHands generalized agent interaction with filesystems, command lines, browsers, execution environments, and benchmarks [2].

The coding agent should therefore be understood as a system composed of at least a model, agent logic, harness, tools, and environment.

2.2 Harnesses

A coding-agent harness mediates how an agent observes and changes its environment.

Recent research has formalized this layer.

Zhong and Zhu identify eleven responsibilities in AI Harness Engineering: task specification, context selection, tool access, project memory, task state, observability, failure attribution, verification, permissions, entropy auditing, and intervention recording [6].

Barbaste et al. analyze eleven production coding harnesses and describe a convergence toward platform-like systems built around execution loops, deterministic retrieval, tools, safety controls, orchestration, and extension mechanisms [7].

A harness primarily answers:

> **How can an agent operate in a software environment?**

The present work introduces a complementary question:

> **Under which engineering conditions should a particular execution be allowed to operate?**

2.3 Context engineering

Coding agents require project-specific information such as architecture, conventions, build procedures, domain concepts, and constraints.

Repository-level instruction files such as AGENTS.md have become a common mechanism for delivering persistent project context.

However, additional context does not automatically improve engineering outcomes.

Gloaguen et al. evaluated repository-level context files across multiple coding agents and found no general improvement in task-success rates, while inference cost increased by more than 20% on average. Context files nevertheless changed agent behavior by encouraging more repository exploration and testing [11].

Context therefore requires engineering decisions concerning relevance, authority, ownership, conflict resolution, and lifecycle.

The problem is not simply:

How much context can the agent receive?

but:

Which context should govern this execution,
with what authority and provenance?

2.4 Structured intent and specification

Specification-driven approaches transform conversational intent into explicit engineering artifacts.

GitHub Spec Kit provides a default **Spec → Plan → Tasks → Implement** process while also supporting extensible workflows, coding-agent integrations, organizational catalogs, and human checkpoints [12].

OpenSpec structures context, artifact rules, and operational guidance. Its configuration is explicitly converted into instructions supplied to the coding agent, illustrating a useful distinction between **policy as context** and **policy as independently enforceable control** [13].

BMAD and GSD similarly introduce progressive context construction, planning stages, specialized workflows, verification activities, and persistent execution artifacts [14], [15].

These systems demonstrate that structured engineering around agents is already an established direction.

2.5 Execution state and workspace

Long-running engineering work may span several agent sessions, human interventions, or physical workspaces.

Execution state is therefore distinct from conversational history.

The AIEF model separates:

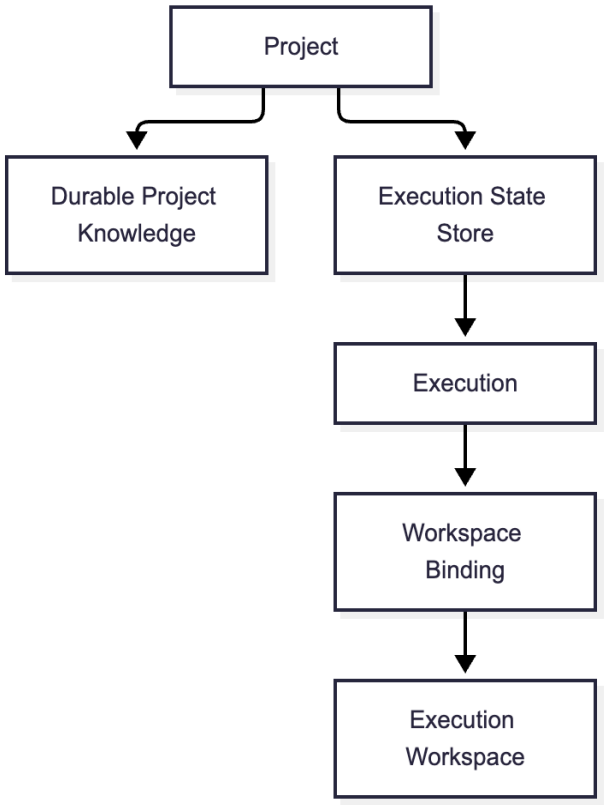


Figure 4. Separation between project knowledge, execution state, and execution workspace.

An execution workspace may be a checkout, worktree, container, VM, or remote development environment. OpenHands similarly exposes workspace as an abstraction across local processes, containers, and remote servers [16].

This distinction enables handoff and resumption without defining engineering identity through an absolute filesystem path.

2.6 Policy and autonomy

Coding-agent capability and authorized autonomy are different concepts.

An agent may technically be capable of modifying infrastructure, pushing commits, or invoking external services while being unauthorized to do so for a particular task.

Conceptually:

$$AuthorizedAutonomy = f(RequestedAutonomy, Policies, Risk)$$

Production coding systems already implement parts of this logic. Kiro applies permissions across platform, administrative, user, and workspace scopes and resolves conflicts restrictively using deny > ask > allow [17].

OpenAI describes Codex deployments using sandbox boundaries, approvals, managed configuration, network controls, and telemetry, with higher-risk actions receiving stronger controls [18].

The novelty question is therefore no longer whether coding agents can be permissioned.

It is how permissions compose with project context, task risk, execution state, verification, and organizational accountability.

2.7 Verification and evidence

Agent-declared success and independently observed success provide different assurance.

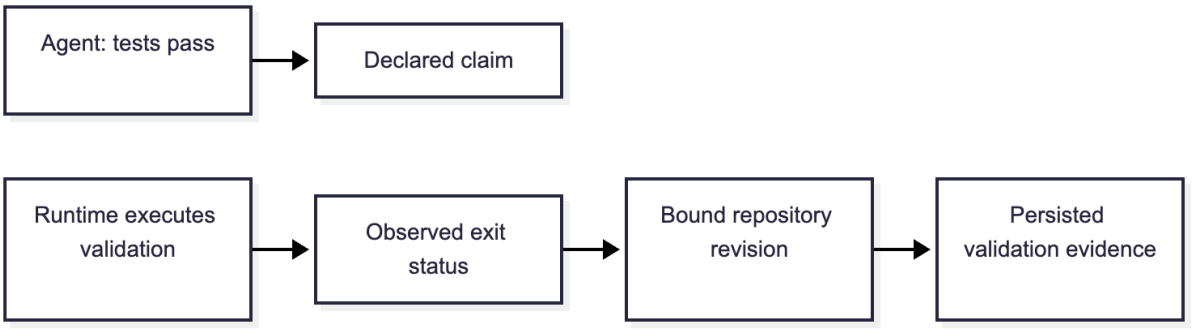


Figure 5. Declared verification versus observed verification.

This distinction also applies to independent review.

A second agent response does not establish independence unless the identities of producer and reviewer can be compared with sufficient assurance.

Evidence should therefore preserve externally meaningful engineering facts rather than private model reasoning.

3. Related Work

3.1 Scope and analysis approach

This section is a **structured related-work analysis**, not a systematic literature review.

The objective is to identify the principal engineering-control surfaces already addressed in research and operational systems and to position the proposed Reference Model relative to them.

The analysis groups related work into four families:

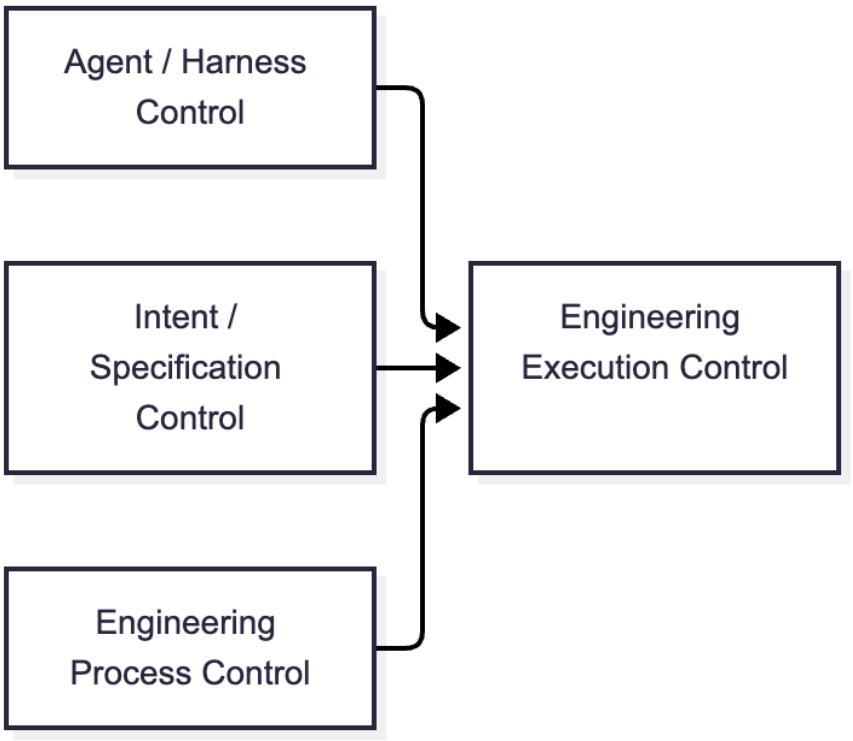


Figure 6. Four control surfaces considered in the related-work analysis.

3.2 Harness engineering

AI Harness Engineering reframes agent reliability as an emergent property of a model–harness–environment system and formalizes task specification, context, memory, state, observability, verification, permissions, and intervention recording as runtime responsibilities [6].

Harness Engineering provides a complementary empirical analysis of production coding-agent runtimes and shows that the harness has evolved from a thin interface into a software platform [7].

Agentic Harness Engineering further investigates automatic harness evolution through observability of components, experience, and decisions, demonstrating that harness design itself can become an optimization object [19].

These works establish the harness as a first-class engineering layer.

AIEF operates one abstraction level higher: its primary object is the governed engineering execution that may use one of several harnesses.

3.3 Specification and workflow systems

Spec Kit structures intent into explicit artifacts and has evolved into an extensible intent-driven harness supporting multiple agents, workflows, catalogs, presets, and extensions [12].

OpenSpec provides explicit schemas, project context, rules, artifacts, and workflow operations but describes many project rules as instructions delivered to the agent [13].

BMAD progressively constructs context across analysis, planning, solutioning, and implementation phases and includes implementation-readiness checks and specialized engineering workflows [14].

GSD focuses strongly on context engineering, persistent state, fresh execution contexts, structured plans, verification, and agent orchestration [15].

These approaches demonstrate that AIEF cannot claim novelty from structured intent, workflow orchestration, or persistent project context alone.

3.4 Process governance

Agentic Agile-V argues that the principal problem has shifted from prompt engineering to engineering-process control. Its SCOPE-V loop—Specify, Constrain, Orchestrate, Prove, Evolve, Verify—combines risk-adaptive workflows, structured artifacts, evidence bundles, independent verification, and human approval [9].

MAGE similarly focuses on trustworthy autonomy through durable engineering structures, bounded action, authority, constraints, sensors, validators, gates, independent evaluation, and retained human authority [10].

These approaches are among the nearest conceptual works to AIEF because they explicitly move engineering authority outside the agent's probabilistic reasoning process.

3.5 Deterministic control planes

Rel(AI)Build proposes a deterministic, tool-agnostic control plane above coding harnesses. It addresses configuration supply chains, permission boundaries, phase gates, auditability, traceability, and compilation of canonical definitions toward several development environments [8]. Its evaluation demonstrates conformance of mechanisms to stated invariants, while developer outcomes remain future work.

This prior work eliminates any credible novelty claim based simply on the phrase **deterministic control plane**.

The distinction pursued here is instead the composition of controls around the **complete engineering execution**.

3.6 Reliability and cost

The Agentic SDLC Control Plane frames agentic software delivery through reliability, verification cost, human attention, operational risk, and production-qualified change [20]. It introduces the notion of a **Verification Tax** and emphasizes that additional agent autonomy may shift rather than eliminate engineering cost.

This perspective directly motivates the fourth research question of this study: governance should be evaluated not only through assurance but also through its computational and human cost.

3.7 Research gap

Existing work provides most of the individual primitives required for governed agentic software engineering:

- context and memory;
- structured specifications;
- workflow orchestration;
- deterministic permissions;
- risk adaptation;
- persistent state;
- human approval;
- independent verification;
- traceability;
- evidence.

However, these mechanisms are typically attached to separate control surfaces: runtime, repository configuration, specification process, methodology, CI pipeline, or organizational governance.

Less is known about how they compose at the level of a **complete Engineering Execution** in which organizational constraints, project knowledge, task-specific risk, execution state, workspace binding, trusted actors, verification, and evidence are governed under one agent-independent control model.

A second gap is empirical.

Architectural conformance can show that a control exists and behaves according to its contract. It does not establish whether increasing control improves compliance, reproducibility, evidence quality, or verification reliability in real coding-agent executions, nor whether the improvement justifies its overhead.

This study addresses both gaps.

4. Research Method

4.1 Research strategy

The study combines **design science research** with controlled empirical evaluation.

Design science is appropriate because the research both constructs an artifact and investigates the artifact's performance in context [21].

The research process is:

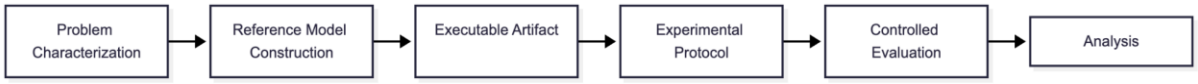


Figure 7. Research process.

Experiment planning follows established empirical software-engineering principles concerning independent variables, repeated observations, confounding factors, measurement, and validity [22].

4.2 Research questions

RQ1 - Engineering-control landscape

> **How are engineering-control responsibilities distributed across coding-agent harnesses, spec-driven frameworks, process frameworks, and deterministic control-plane approaches?**

RQ2 - Architectural separation

> **How can deterministic engineering control be separated from probabilistic coding-agent reasoning while remaining independent of the underlying coding agent?**

RQ3 - Effectiveness

> **How does increasing engineering control affect policy compliance, reproducibility, traceability, evidence completeness, and verification reliability in coding-agent executions?**

RQ4 - Cost of control

What execution, verification, and human-attention overhead is introduced by increasing levels of engineering control?

4.3 Hypotheses

H1 - Policy compliance

Increasing deterministic engineering control will reduce policy violations compared with less structured coding-agent execution.

H2 - Reproducibility

Increasing control over context, policy, and execution state will reduce variability between repeated executions of equivalent software-engineering tasks.

H3 - Evidence and verification

Governed executions will produce more complete and independently verifiable execution evidence than direct or context-only agent executions.

H4 - Cost of control

Increasing engineering control will introduce measurable execution, verification, and/or human-attention overhead.

Exploratory hypothesis

The relative value of stronger engineering control increases with the engineering risk of the task.

This final hypothesis remains exploratory unless the task corpus supports a sufficiently robust pre-defined risk stratification.

4.4 Artifact freeze

The executable research artifact is frozen before outcome analysis to reduce the risk of adapting the implementation to favorable experimental results.

The frozen baseline is:

AIEF version: 1.0.0
API/schema: aief.dev/v1alpha1

The baseline confirms agent/provider independence of Core semantics, deterministic layered policy resolution, fingerprinted governed plans, explicit execution state and workspace semantics, revision-bound verification evidence, trust-derived reviewer independence, and explicit enforcement assurance.

Linked and managed Git worktrees use canonical project execution state; leases remain Execution-scoped and evidence survives managed-worktree deletion.

No behavioral modification should be introduced solely because experimental outcomes are unfavorable.

5. AIEF Reference Model

5.1 Central principle

The model separates two planes.

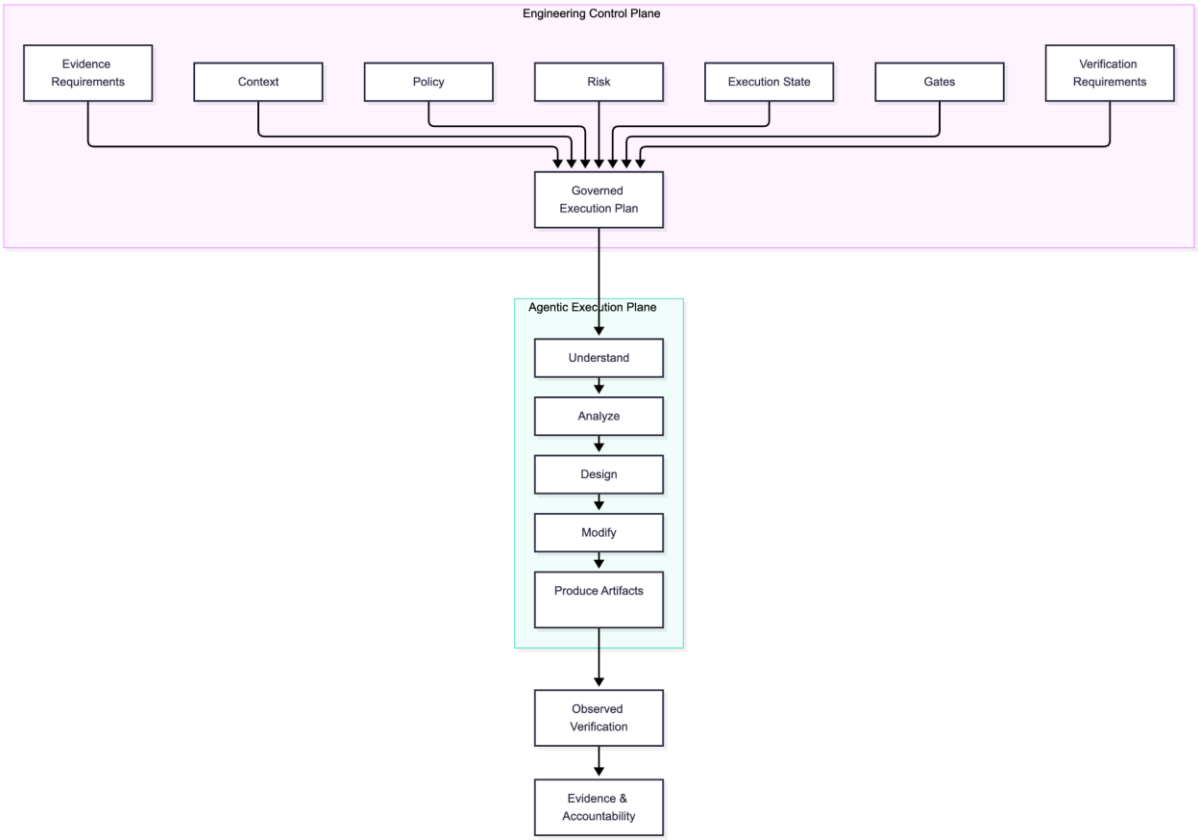


Figure 8. AIEF two-plane architecture.

The Engineering Control Plane answers:

Under which engineering conditions may this execution proceed?

The Agentic Execution Plane answers:

How should the software-engineering problem be understood and solved?

5.2 Core entities

The Reference Model defines the following principal entities:

Engineering System Definition. Reusable workflows, policies, standards, contracts, schemas, and governance semantics independent of any individual project.

Organizational Constraints. Organization-specific policies, risk constraints, approval requirements, and reusable engineering practices.

Project Engineering Context. Durable project-owned architecture, domain knowledge, repository topology, conventions, decisions, and approved knowledge.

Engineering Task. Normalized engineering intent independent of the agent and execution attempt.

Governed Execution. One controlled realization of a Task.

Governed Execution Plan. Execution-specific deterministic control contract.

Execution Workspace. Mutable environment in which software work occurs.

Agentic Executor. Replaceable probabilistic software-reasoning actor.

Verification. Mechanisms establishing observable facts about execution outcomes.

Evidence. Persisted externally meaningful facts connecting intent, action, validation, and decision.

Human Accountability. Explicit human authority where required by risk or policy.

5.3 Governed lifecycle

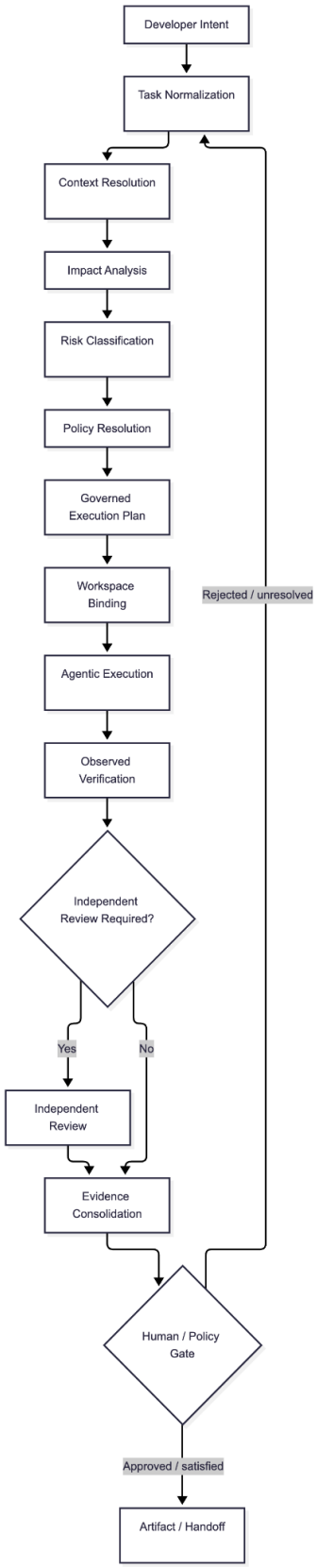


Figure 9. Governed Engineering Execution lifecycle.

5.4 Governed Execution Plan

Conceptually:

$$GEP = f(T, C, I, R, P, W)$$

where:

- (T): Task;
- (C): Effective Engineering Context;
- (I): Impact;
- (R): Risk;
- (P): Policies;
- (W): Workflow.

The resulting control contract contains conceptually:

GEP=⟨scope, workflow, autonomy, permissions, gates, verification, evidence⟩

The GEP converts reusable engineering constraints into execution-specific control.

The AIEF implementation persists a fingerprinted plan bound to task, execution, workflow, policy revision, scope, permissions, gates, risk, autonomy, and review requirements.

5.5 Layered governance

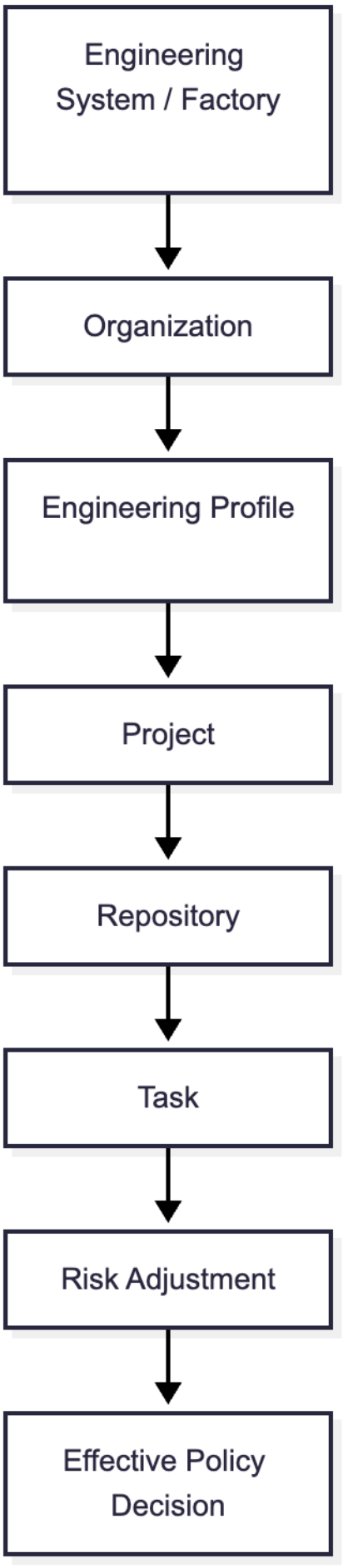

Engineering
System / Factory
Organization
Engineering Profile
Project
Repository
Task
Risk Adjustment
Effective Policy
Decision

Figure 10. Layered policy resolution.

For autonomy:

$$A_{\text{eff}} = \min\left(A_{\text{req}}, A_{\text{risk}}, \min_{i \in L} A_i\right)$$

For risk:

$$R_{\text{eff}} = \max_{i \in L} R_i$$

For required gates:

$$G_{\text{eff}} = \bigcup_{i \in L} G_i$$

A lower-authority layer may strengthen control but cannot silently weaken mandatory higher-authority constraints.

5.6 Knowledge ownership

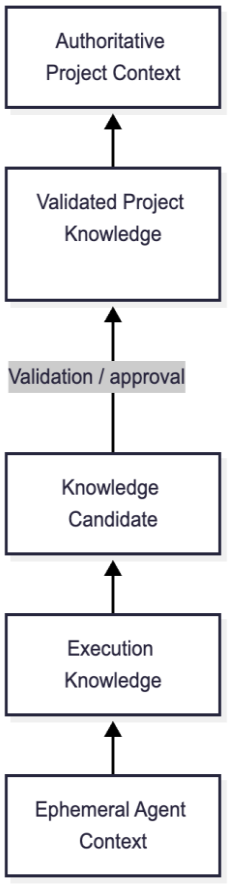


Figure 11. Controlled knowledge promotion.

The generic Engineering System Definition does not become a global learned memory containing facts from individual projects.

Project-specific knowledge remains project-owned.

5.7 Enforcement assurance

Policy resolution and policy enforcement are separate concepts.

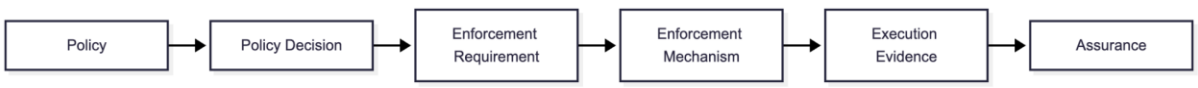


Figure 12. Enforcement assurance chain.

The model distinguishes:

- ENFORCED;
- ENFORCEABLE;
- UNVERIFIED;
- UNAVAILABLE.

A mandatory requirement fails closed when the required assurance cannot be established.

5.8 Actor assurance and review

Actor identity may be:

- DECLARED;
- OBSERVED;
- TRUSTED;
- UNVERIFIED.

Review independence may be:

- INDEPENDENT;
- NOT_INDEPENDENT;
- UNVERIFIED;
- NOT_REQUIRED.

Conceptually:

$$\text{IndependentReview} = \text{Trusted}(\text{Producer}) \land \text{Trusted}(\text{Reviewer}) \land (\text{Producer} \neq \text{Reviewer})$$

A review artifact cannot establish its own independence solely by declaration.

5.9 Model invariants

The Reference Model preserves the following invariants:

1. Agent independence.
2. Provider independence.
3. Project ownership of project knowledge.
4. Deterministic control where deterministic evidence exists.
5. Isolation of probabilistic reasoning from control authority.
6. Restrictive governance composition.
7. Explicit bounded autonomy.
8. Evidence before claims.
9. Workspace-path independence.
10. Separation of execution state from execution workspace.
11. Controlled knowledge promotion.
12. Explicit human accountability.
13. Fail-closed mandatory controls.
14. Reproducibility metadata sufficient to compare executions.

6. Reference Implementation

6.1 Implementation role

AIEF is an executable realization of the Reference Model.

The implementation is not itself the scientific model.

Implementation choices such as Python modules, YAML schemas, Git worktrees, filesystem locations, or adapter formats are therefore treated as replaceable realization mechanisms.

6.2 Logical architecture

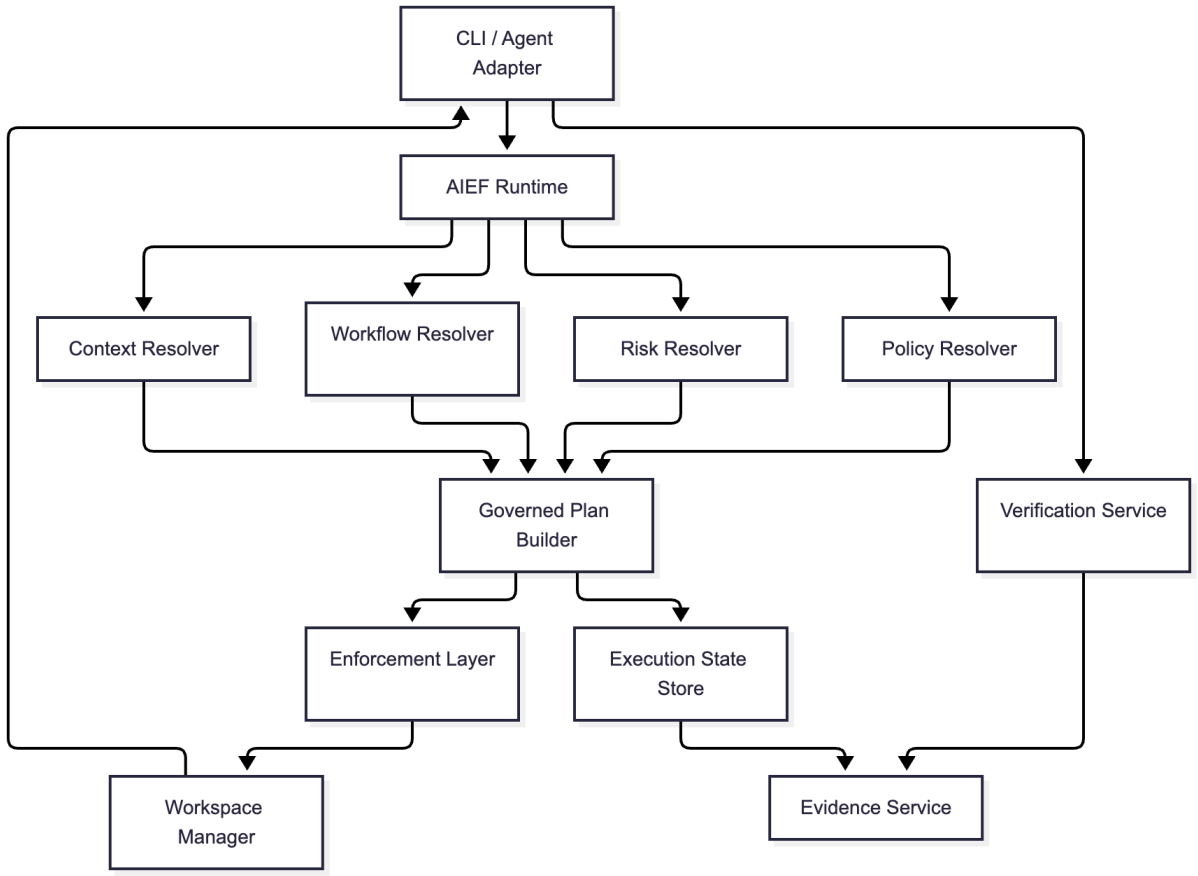


Figure 13. Logical AIEF implementation architecture.

The Runtime remains the deterministic coordination boundary. Coding agents perform semantic reasoning and software modification through adapters rather than being embedded as reasoning logic in the Core.

6.3 Effective Engineering Context

The implementation resolves context through increasingly specific layers:

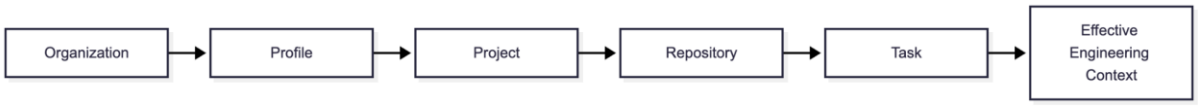


Figure 14. Effective Engineering Context resolution.

This enables organization-wide practices without copying project-specific knowledge into the Factory.

6.4 Execution state

AIEF separates Project truth, Execution state, and physical workspace.

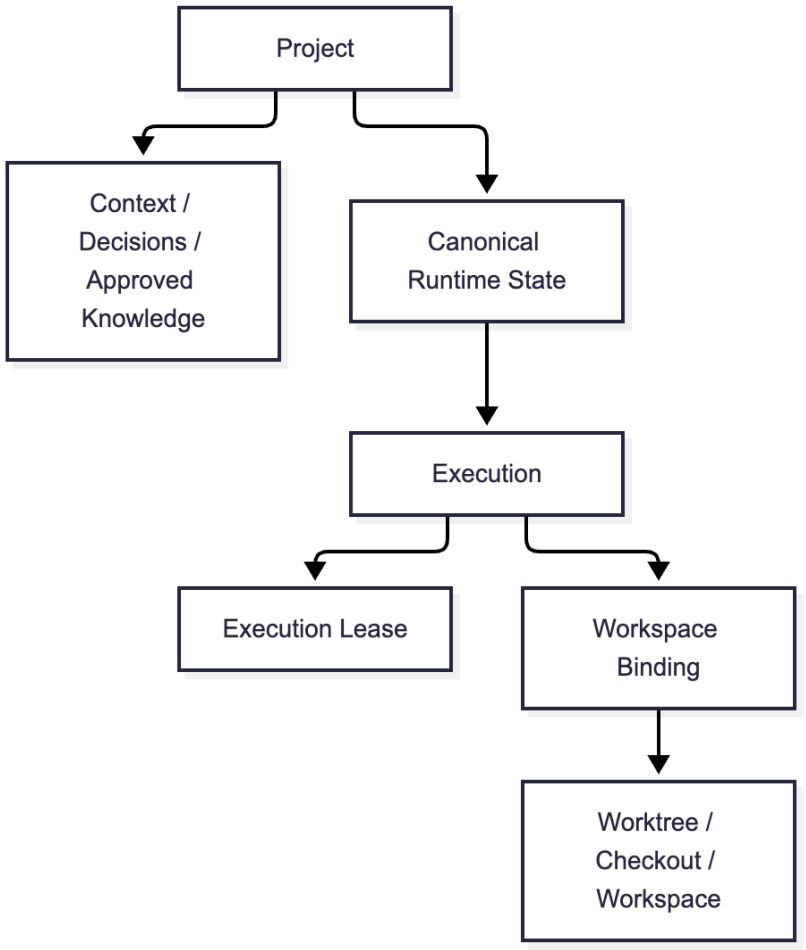


Figure 15. AIEF execution-state ownership.

The frozen baseline classifies worktree handling as WORKTREE_SAFE_WITH_EXPLICIT_LIMITATION: linked worktrees share canonical execution state, while deletion of the entire canonical local checkout remains outside the supported recovery boundary.

6.5 Evidence

A successful governed execution can connect:

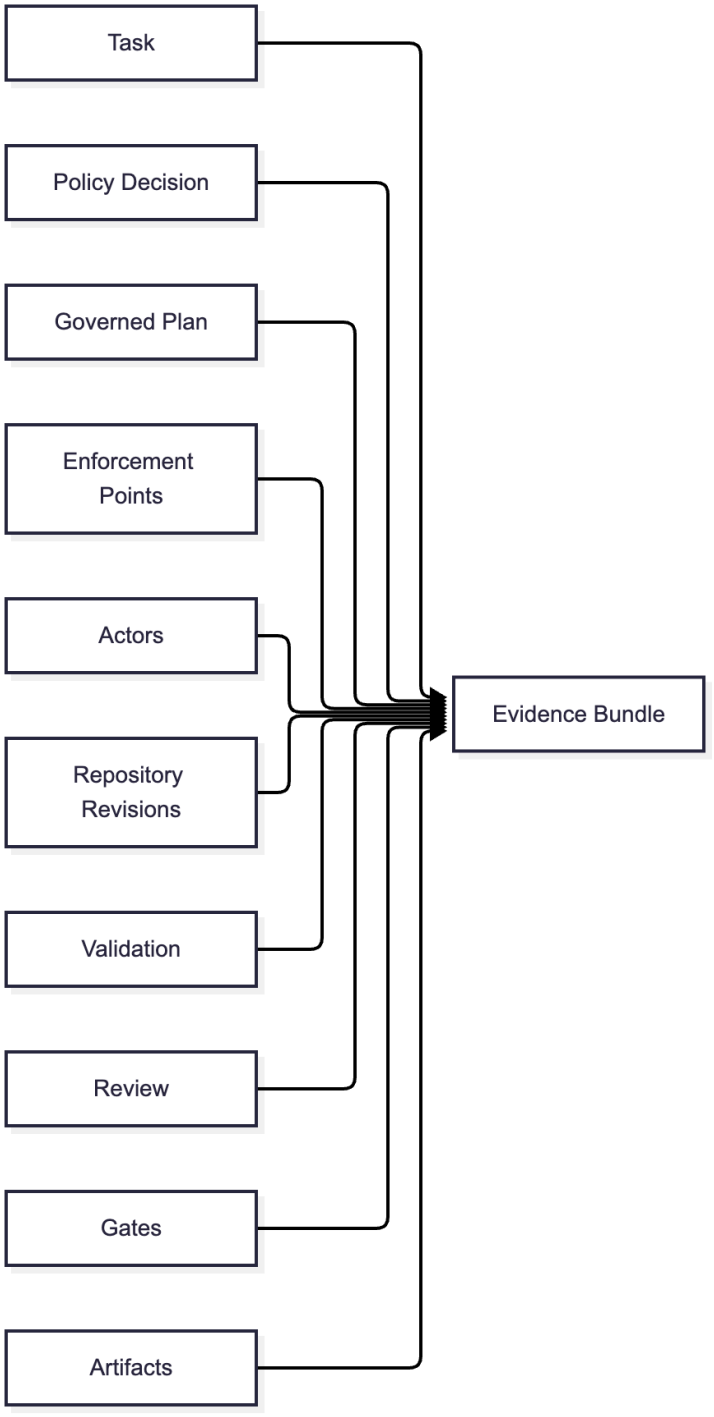


Figure 16. EvidenceBundle composition.

The bundle uses stable identifiers and fingerprints and preserves explicit unavailable states instead of converting missing telemetry into successful or zero-valued observations.

6.6 Heterogeneous agents

AIEF is designed to preserve engineering semantics when changing the underlying coding agent.

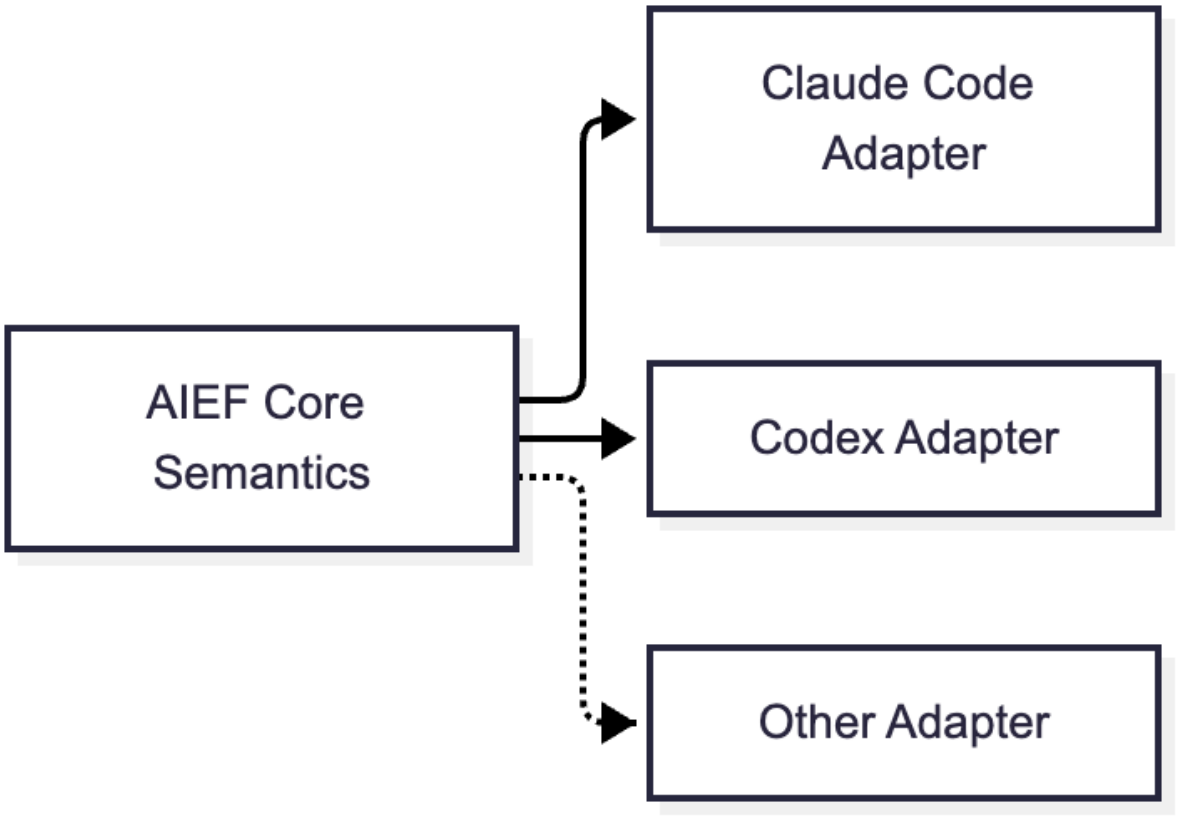


Figure 17. Agent-independent Core with replaceable adapters.

The frozen baseline validates common Core workflow, policy, plan, and evidence semantics through Claude Code and Codex conformance scenarios.

6.7 Baseline

The frozen experimental implementation is:

AIEF: 1.0.0
API: aief.dev/v1alpha1

This validates the implementation contracts.

It does **not** establish that AIEF improves software quality, productivity, reproducibility, compliance, or cost effectiveness.

7. Experimental Design

7.1 Independent variable

The principal independent variable is the **level of engineering control**.

Four experimental conditions are defined.

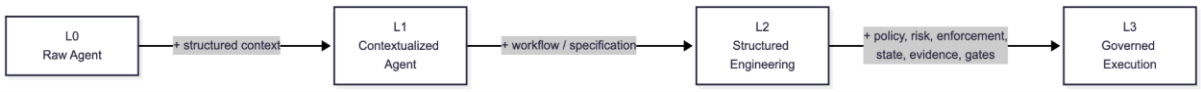


Figure 18. Experimental control conditions.

L0 - Raw Agent

Developer request → Coding agent

Only the minimum technical environment required for execution is provided.

L1 - Contextualized Agent

Developer request
+ structured project context
→ Coding agent

L2 - Structured Engineering

Intent
→ specification / structured workflow
→ agent execution
→ structured verification

L3 - Governed Engineering Execution

Intent

→ effective context
→ impact
→ risk
→ policy
→ governed plan
→ controlled execution
→ verification
→ evidence
→ gates

L0–L3 are experimental conditions, not universal maturity levels.

7.2 Experimental unit

One experimental unit is one coding-agent execution of one selected engineering task under one control condition.

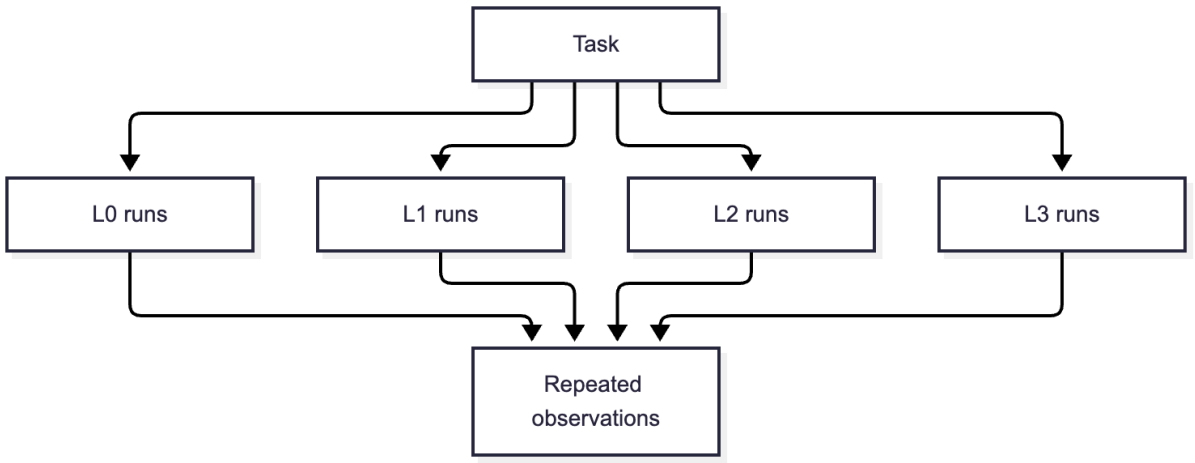


Figure 19. Repeated-measure experimental structure.

Repeated executions are required because agent behavior is probabilistic.

7.3 Controlled factors

Where technically possible, comparable runs should preserve:

- task definition;
- starting repository revision;
- acceptance criteria;
- model;
- coding agent;

- environment;
- dependency versions;
- available tools;
- network conditions;
- evaluation rules.

When multiple agents or models are included, these become explicit factors rather than being conflated with the engineering-control level.

7.4 Outcome measures

RQ3 considers:

Policy compliance

Possible metrics include:

$$\text{ComplianceRate} = \frac{SatisfiedMandatoryPolicies}{ApplicableMandatoryPolicies}$$

and:

$$\text{ViolationRate} = \frac{ObservedPolicyViolations}{ApplicablePolicyChecks}$$

Reproducibility

Reproducibility concerns engineering outcomes rather than identical generated source.

Comparisons may include:

- acceptance-criteria consistency;
- validation-result consistency;
- scope variability;
- policy-decision consistency;
- evidence completeness;
- execution-path variability.

Traceability

Traceability may be evaluated through the availability of links among:



Figure 20. Minimal traceability chain.

Evidence completeness

A pre-defined evidence checklist should determine whether required execution facts are:

- present;
- verified;
- unavailable;
- missing.

Verification reliability

The analysis should distinguish:

agent-declared success

from:

independently observed validation

7.5 Cost-of-control measures

RQ4 separates multiple forms of overhead.

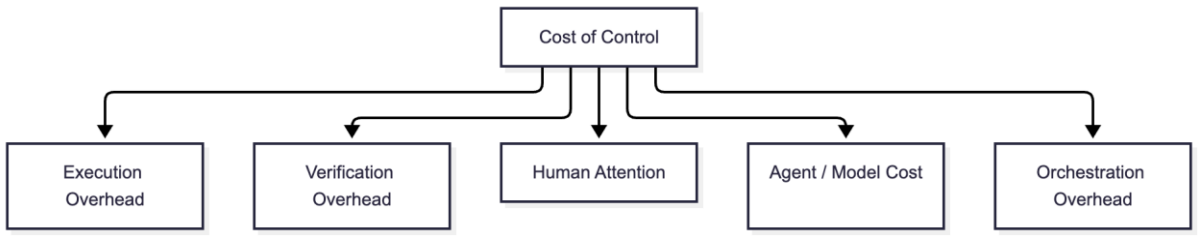


Figure 21. Cost-of-control decomposition.

The frozen baseline directly observes:

- total execution duration;
- Runtime duration;
- validation duration;
- phase duration;
- gate-wait duration.

Agent-active duration, provider duration, token counts, and monetary cost remain unavailable unless trustworthy telemetry is supplied.

Unavailable data must not be reconstructed from unreliable proxies.

7.6 Reproducibility metadata

Each run should record, where applicable:

experimentalCondition
repeatIndex
environmentIdentifier
agentIdentifier
modelIdentifier
adapterIdentifier
AIEFVersion
AIEFCommit
projectCommit
startingRepositoryCommit
workflowVersion
contextFingerprint
policyFingerprint
governedPlanFingerprint

Experimental condition is explicit metadata and is never inferred from Runtime autonomy.

7.7 Planned analysis

The final statistical analysis will be chosen after the experimental protocol and pilot confirm the structure and distribution of collected observations but before analysis of the complete outcome dataset.

The analysis should report both **statistical differences** and **engineering effect sizes**, where appropriate.

Single successful executions will not be treated as representative evidence.

7.8 Experimental pipeline

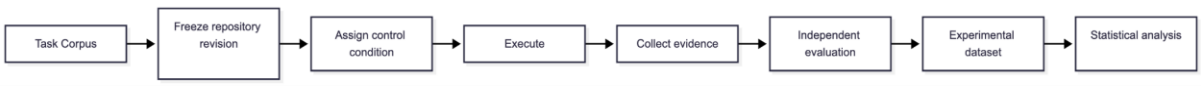


Figure 22. Planned experimental pipeline.

8. Results

8.1 RQ3 - Policy compliance

[Reserved for experimental evidence.]

8.2 RQ3 - Reproducibility

[Reserved for experimental evidence.]

8.3 RQ3 - Traceability and evidence completeness

[Reserved for experimental evidence.]

8.4 RQ3 - Verification reliability

[Reserved for experimental evidence.]

8.5 RQ4 - Execution overhead

[Reserved for experimental evidence.]

8.6 RQ4 - Verification overhead

[Reserved for experimental evidence.]

8.7 RQ4 - Human-attention overhead

[Reserved for experimental evidence.]

8.8 Cross-agent observations

[Reserved for experimental evidence.]

9. Discussion

9.1 Architectural implications

Independently of empirical performance outcomes, the Reference Model highlights an architectural distinction between **coding-agent capability** and **engineering authority**.

An agent may possess the technical capability to execute an action without possessing organizational authority to perform it.

This leads to:

Capability ≠ Authorization

Likewise:

AgentClaim ≠ EngineeringEvidence

and:

Workspace ≠ ExecutionState

These distinctions suggest that scaling agentic development requires architecture around agents rather than only increasingly capable agents.

The Reference Model also indicates that engineering governance does not need to replace agent reasoning.

Instead:

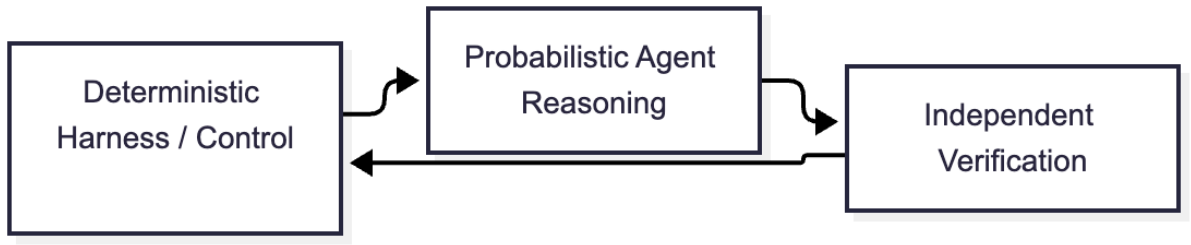


Figure 23. Control, reasoning, and verification as complementary mechanisms.

9.2 Relationship to harness engineering

Harness Engineering asks how models should interact reliably with tools and environments.

AIEF treats the harness as one execution capability underneath a broader engineering-control abstraction.

The relationship is therefore complementary rather than competitive:



becomes part of:

Governed Engineering Execution

This explains why many harness-level innovations—permissions, context compaction, tool mediation, state, sandboxing—remain useful without being duplicated in the Reference Model.

9.3 Relationship to specification-driven development

Specification-driven systems reduce ambiguity by converting intent into explicit artifacts.

AIEF extends this idea from:



toward:

Intent
→ Context
→ Specification / Workflow
→ Risk
→ Policy
→ Authorized Execution
→ Verification
→ Evidence

Specification is therefore one component of engineering control, not the complete control model.

9.4 Relationship to deterministic control planes

Rel(AI)Build demonstrates that deterministic governance should not be delegated entirely to probabilistic agent reasoning [8].

The AIEF contribution should consequently not be framed as:

> introducing a deterministic control plane.

Its distinct research object is:

> **the complete governed engineering execution as the unit over which deterministic control, probabilistic reasoning, state, verification, and accountability are composed.**

9.5 Organizational implications

If the empirical results eventually support the model's hypotheses, engineering organizations may need to evolve beyond developer-specific agent configuration.

The organizational trajectory could be conceptualized as:

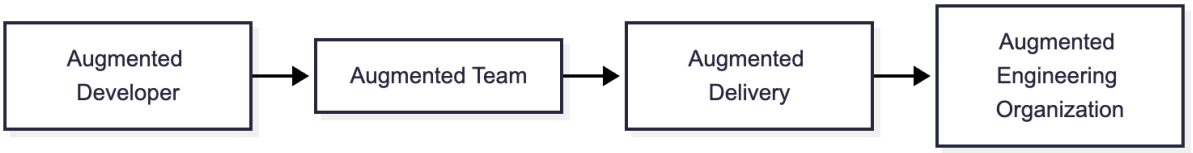


Figure 24. Organizational scaling trajectory for agentic engineering.

The transition would involve moving reusable engineering authority from personal prompts toward shared standards, policies, workflows, evidence requirements, and controlled execution mechanisms.

This interpretation must ultimately be conditioned by the experimental cost-benefit results.

9.6 Empirical interpretation

[Reserved until experimental evidence is available.]

RQ3 interpretation

[Reserved.]

RQ4 interpretation

[Reserved.]

Hypothesis outcomes

[Reserved.]

Unexpected observations

[Reserved.]

10. Threats to Validity

10.1 Construct validity

Concepts such as governance, reproducibility, compliance, evidence completeness, and verification reliability are broader than any single metric.

The study therefore requires operational definitions to be fixed before outcome analysis.

For example, reproducibility must not be reduced to textual or code identity. Different implementations may satisfy the same engineering obligations.

Similarly, an evidence artifact should not automatically count as evidence completeness merely because a file exists; its provenance and required fields must be considered.

10.2 Internal validity

Several variables could create apparent differences between L0–L3 independently of engineering control.

Potential confounders include:

- changing model versions;
- agent-runtime updates;
- repository revisions;
- dependency changes;
- different tool permissions;
- different network access;
- machine performance;
- cache effects;
- nondeterministic external services.

The experiment therefore records environment and revision metadata and should execute comparable conditions from equivalent starting states.

10.3 Treatment leakage

The conditions must differ only through intended engineering-control mechanisms.

A particular risk is accidentally supplying L0 with project context that effectively reproduces L1, or implementing L2 through AIEF mechanisms that already expose L3 controls.

Each condition therefore requires a documented treatment contract.

10.4 Agent stochasticity

Coding agents are probabilistic.

Repeated observations are required.

A single success or failure cannot establish the effect of a control level.

10.5 Implementation bias

AIEF is both the research artifact and an implementation developed by the researcher.

This introduces confirmation risk.

Mitigations include:

- freezing the research artifact before complete outcome analysis;
- specifying hypotheses in advance;
- preserving negative results;
- avoiding behavioral fixes motivated by experimental underperformance;
- separating architectural conformance claims from empirical-effectiveness claims.

10.6 Baseline asymmetry

L3 is implemented explicitly through AIEF, whereas L1 and L2 may use lighter mechanisms or external frameworks.

Care is required to ensure that the comparison measures **control characteristics**, not simply differences in engineering effort invested into each implementation.

The study should therefore describe each condition through capabilities rather than product identity.

10.7 External validity

A finite set of repositories, tasks, languages, agents, and models cannot represent all software-development contexts.

Results may not generalize directly to:

- very large monorepositories;
- embedded systems;
- safety-critical development;
- regulated financial software;
- highly interactive frontend work;
- greenfield development;
- legacy modernization;
- infrastructure operations.

Replication across task classes will therefore remain necessary.

10.8 Temporal validity

Coding-agent platforms evolve rapidly.

Capabilities available during the experiment may change after publication.

For this reason, the study records explicit versions and treats the Reference Model as the primary conceptual contribution rather than the exact capabilities of one vendor runtime.

10.9 Conclusion validity

Sample size, variance, repeated-measure structure, and metric distributions will determine the appropriate statistical analysis.

Effect sizes and confidence intervals should accompany significance testing where applicable.

Absence of statistical evidence should not be reformulated as proof of equivalence.

11. Conclusion

Coding agents are changing the unit of automation in software engineering.

The transition is no longer only from manually written code to AI-generated code. It is increasingly a transition from human-executed development activities toward **agent-executed engineering trajectories**.

This shift creates a new systems problem.

Context, permissions, specifications, workflow, validation, human approval, and evidence can no longer remain isolated mechanisms if autonomous execution is expected to scale across teams and organizations.

This paper proposes the **AIEF Reference Model**, which treats the Governed Engineering Execution as the central unit around which these mechanisms are composed.

The model separates:

probabilistic reasoning

from:

deterministic engineering control

while preserving explicit relationships among project knowledge, organizational policy, risk, autonomy, execution state, workspace, verification, evidence, and human authority.

AIEF demonstrates that this architecture can be implemented while remaining independent from the underlying coding agent at the Core semantics level. Its frozen experimental baseline provides deterministic policy resolution, fingerprinted governed plans, explicit enforcement assurance, revision-bound verification, trusted actor semantics, canonical execution state, and structured evidence across heterogeneous agent adapters.

These architectural properties do not establish that greater engineering control produces better software outcomes.

That question remains empirical.

The planned study therefore compares increasing levels of engineering control and evaluates both their potential benefits and their **cost of control**.

[Final experimental conclusion to be inserted after results are available.]

The broader proposition of this work is that the future reliability of coding agents may depend less on treating each new model as a more autonomous developer and more on engineering the system in which agent autonomy is exercised.

References

[1] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-Agent: Agent-Computer Interfaces Enable Automated Software Engineering," *NeurIPS*, 2024.

[2] X. Wang et al., "OpenHands: An Open Platform for AI Software Developers as Generalist Agents," arXiv:2407.16741, 2024.

[3] R. Robbes, T. Matricon, T. Degueule, A. Hora, and S. Zacchiroli, "Agentic Much? Adoption of Coding Agents on GitHub," *ACM Transactions on Software Engineering and Methodology*, 2026, doi:10.1145/3822180.

[4] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can Language Models Resolve Real-world GitHub Issues?," *ICLR*, 2024.

[5] A. E. Hassan et al., "Agentic Software Engineering: Foundational Pillars and a Research Roadmap," arXiv:2509.06216.

[6] H. Zhong and S. Zhu, "AI Harness Engineering: A Runtime Substrate for Foundation-Model Software Agents," arXiv:2605.13357, 2026.

[7] P. Barbaste, T. Darrigol, G. Vu, and T. Wiltberger, "Harness Engineering: Anatomy, Architecture, and Evolution of Coding Agents — A Source-Code Study of Eleven Systems," arXiv:2609.00006, 2026.

[8] P. Madatha, "A Deterministic Control Plane for LLM Coding Agents," arXiv:2606.26924, 2026.

[9] C. Koch, "Agentic Agile-V: From Vibe Coding to Verified Engineering in Software and Hardware Development," arXiv:2605.20456, 2026.

[10] J. C. Davis, K. Kalu, H. Peng, and P. V. Patil, "Model-Based Agentic Software Engineering," arXiv:2608.25174, 2026.

[11] T. Gloaguen, N. Mündler, M. N. Müller, V. Raychev, and M. Vechev, "Evaluating AGENTS.md: Are Repository-Level Context Files Helpful for Coding Agents?," 2026.

[12] GitHub, "Spec Kit Documentation," 2026.

[13] OpenSpec, "Project Configuration," 2026.

[14] BMAD Method, "Workflow Map," 2026.

[15] GSD, "Context Engineering and Spec-Driven Development," 2026.

[16] OpenHands, "Workspace Architecture," 2026.

[17] Kiro, "Enterprise Permission Policies," 2026.

[18] OpenAI, "Running Codex Safely at OpenAI," 2026.

[19] J. Lin et al., "Agentic Harness Engineering: Observability-Driven Automatic Evolution of Coding-Agent Harnesses," arXiv:2604.25850, 2026.

[20] H. Bhati, "Beyond Code Generation: Reliability, Verification, and Cost Economics in the Agentic Software Development Lifecycle," arXiv:2609.04681, 2026.

[21] R. J. Wieringa, *Design Science Methodology for Information Systems and Software Engineering*, Springer, 2014.

[22] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*, Springer, 2012.